

函数式编程核心概念



函数是一等公民

北京理工大学计算机学院
金旭亮

“函数是一等公民”



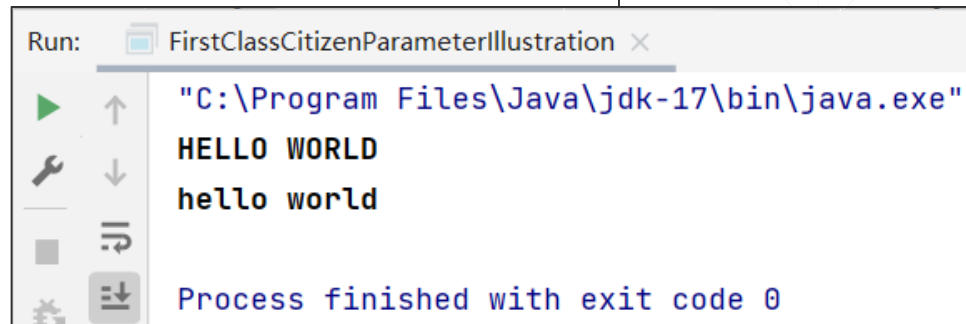
在函数式编程代码中，函数与其它数据类型一样，也可以进行“赋值”和“传送”，具体来说，就是可以定义“函数类型”的变量，函数可以成为另一个函数的“参数”，函数也可以返回“另一个函数”.....，

Java通过“函数式接口”+ Lambda表达式，实现函数式编程。

```
public class FunctionalInterface {  
    public static void main(String[] args) {  
        //定义一个函数式接口变量，接收一个Lambda表达式  
        Function<Integer, Integer> doubleValue = value -> value * 2;  
        //执行Lambda表达式，输出：200  
        System.out.println(doubleValue.apply(100));  
    }  
}
```

函数作为另一个函数的参数.....

```
public class FirstClassCitizenParameterIllustration {  
    //一个接收一个函数的静态方法  
    public static String concatAndTransform(String a,String b,  
        Function<String, String> stringTransform) {  
        if(stringTransform!=null){  
            a=stringTransform.apply(a);  
            b=stringTransform.apply(b);  
        }  
        return a+b;  
    }  
  
    public static void main(String[] args) {  
        //可以使用Lambda表达式直接作为上述静态方法的参数  
        System.out.println(concatAndTransform("Hello ", "World", (s)->s.toUpperCase()));  
        System.out.println(concatAndTransform("Hello ", "World", (s)->s.toLowerCase()));  
    }  
}
```



```
public class FirstClassCitizenVariableIllustration {  
  
    private static String concatStrings(String a,String b) {  
        return a+b;  
    }  
    private String concatStrings2(String a,String b) {  
        return a+b;  
    }  
    public static void main(String[] args) {
```

可以定义函数式接口类型的变量，并且使用Lambda表达式（或方法引用）给它赋值。

//将一个Lambda表达式赋给一个函数式接口类型的变量

```
BiFunction<String, String, String> concatFunction=(s,t)->{  
    return s+t;  
};
```

//直接调用这个变量

```
System.out.println(concatFunction.apply("Hello ", "World 1"));
```

//让变量引用一个满足条件的静态方法

```
concatFunction=FirstClassCitizenVariableIllustration::concatStrings;  
System.out.println(concatFunction.apply("Hello ", "World 2"));
```

方法引用

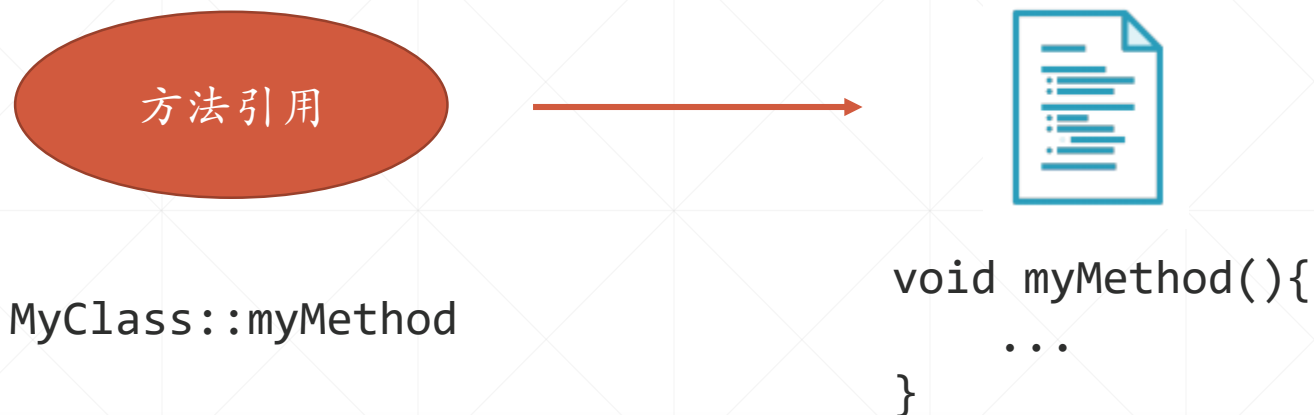
//让变量引用一个满足条件的实例方法

```
FirstClassCitizenVariableIllustration obj=new FirstClassCitizenVariableIllustration();  
concatFunction=obj::concatStrings2;  
System.out.println(concatFunction.apply("Hello ", "World 3"));
```

```
}
```

```
}
```

方法引用—给方法一个“名字”

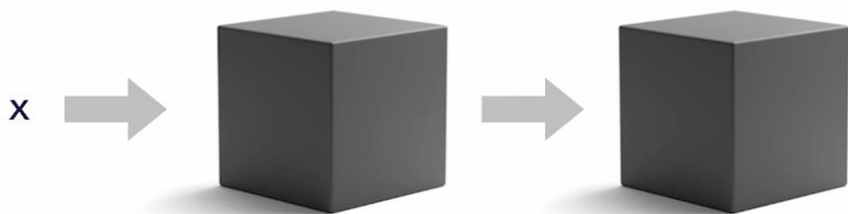


方法引用实例，生成10个随机数并输出：

```
Stream.generate(Math::random)
    .limit(10)
    .forEach(System.out::println);
```

println成为了forEach方法的参数，只要满足特定的需求，forEach可以接收各种各样的方法引用。

高阶函数



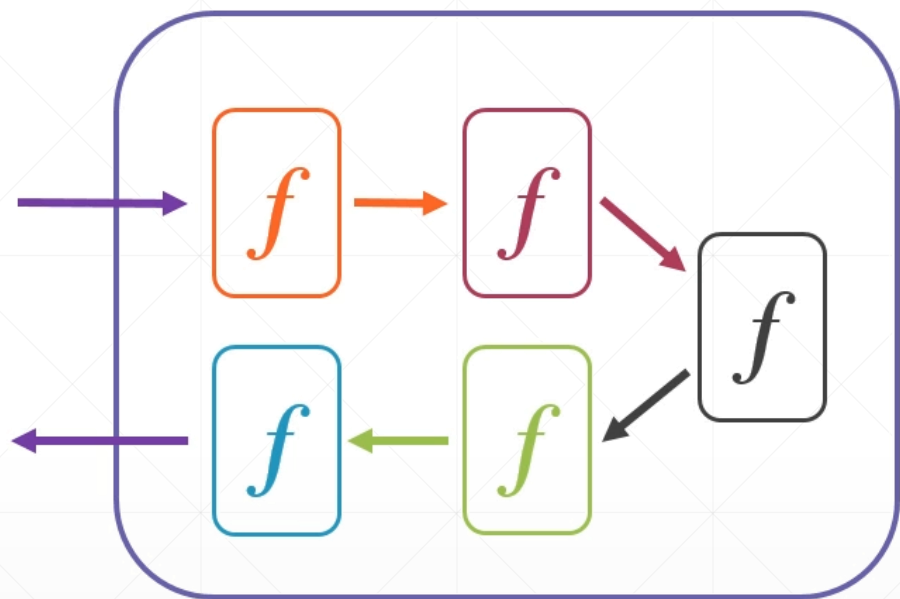
高阶函数：返回函数的函数，它也可以接收函数作为参数。

```
public class HighOrderFunctionIllustration {  
    //返回函数 (即Lambda表达式) 的函数, 称为“High Order Function”  
    private static Supplier<String> createCombineAndTransform(  
        final String a, final String b,  
        final Function<String, String> transformer) {  
        //注意: 这里把一个Lambda表达式return出去了.....  
        return () -> {  
            String aa = a;  
            String bb = b;  
            if (transformer != null) {  
                aa = transformer.apply(a);  
                bb = transformer.apply(b);  
            }  
            return aa + bb;  
        };  
    }  
  
    public static void main(String[] args) {  
        //传入一个方法引用作为参数, 又得到一个Supplier函数对象  
        Supplier<String> xformOperation = createCombineAndTransform(  
            a: "Hello ", b: "World", String::toUpperCase);  
        //调用Supplier接口所定义的get方法, 获取最终结果  
        System.out.println(xformOperation.get());  
    }  
}
```

由于高阶函数可以返回函数，执行这个返回的函数，又可以得到另外一个函数，.....，由此可以一直持续下去，从而很容易地实现“级联”调用。

JDK中的Stream API，里面就有许多这样子的高阶函数。

使用高阶函数构建数据加工“流水线”.....



采用函数式编程范式处理数据，体现为使用一组可动态组合、级联调用的高阶数据处理函数操控数据集中的元素，可分为三大类：

Map（转换）

filter（过滤）

reduce（归并）



JDK 8所引入的Stream API，适用于JDK中的所有集合组件，能很方便地构建出一条数据加工流水线，基于它所写出来的数据处理代码，具有典型的函数式编程风格。

示例

使用Stream API以级联调用的方式，筛选出集合中的所有偶数.....

```
public class UseStreamAPI {  
    public static void main(String[] args) {  
        //原始数据集合  
        int[] arr = {1, 2, 3, 4, 5, 6, 7, 8, 9};  
        //构建Stream对象  
        var nums : IntStream = Arrays.stream(arr);  
        //使用Stream API构建的数据处理流水线  
        var evenNumbers : String = nums  
            //过滤  
            .filter(num -> num % 2 == 0) IntStream  
            //装箱为Integer流  
            .boxed() Stream<Integer>  
            //Integer转换为字符串  
            .map(Object::toString) Stream<String>  
            //收集所有字符串，以“,”分隔，连接起来，构建一个字符串  
            .collect(Collectors.joining( delimiter: ", "));  
        //输出: 2, 4, 6, 8  
        System.out.println(evenNumbers);  
    }  
}
```

函数作为值来使用



- ▶ 函数可以赋值给一个变量
- ▶ 函数可以作为另一个函数的参数
- ▶ 函数可以作为另一个函数的返回值

小结：Java对函数式编程的支持



Java对“函数式编程”范式的支持，是从JDK 8引入Lambda表达式之后才开始的，主要包容以下几个部分：

- 1 Lambda表达式与函数式接口特性（JDK 8起）
- 2 Stream API（JDK 8起）与Flow API（JDK 9起）
- 3 对JDK原有组件进行改造，以支持函数式编程范式



Java生态圈中，很早就开始应用“函数式编程”了，并且出现了许多第三方的库或框架，比如RxJava，kotlin协程，Spring WebFlux等等，它们在实际开发中得到了广泛的应用。